

CSCI 493-81

Lab 9

Security Lab

Topic: Exploits

Prof. Dietrich

April 23, 2025

Group size: 3-4 — Setup needed: Windows/macOS Unix hosts, SPHERE.^{1,2}

1 General description

In this lab, we are experimenting with exploits, such as buffer overflows. This lab is based on a SPHERE exercise by Peter A. H. Peterson and Peter Reiher.

2 File with samples and results

The files are in lab-exploits.zip.

3 Setting up for the experiment

1. Login to SPHERE.
2. Create an instance of the exercise “bufferoverflow.”

Make sure that you save your work as you go. See the instructions in the submission section of this exercise for information about save and restore scripts. Save any changes you make to the source code, your patches, memos, etc. in your home directory.

3.1 Requirement

- You use the instructions provided with this lab. You should use SPHERE, as this experiment can do damage.
- Your output should demonstrate that you have done the experiments, such as screenshots, Unix typescripts, and the traffic logs showing network packets.

¹This document is Copyright ©2025 Sven Dietrich

²This document is for internal CUNY use only

4 Tasks

4.1 Buffer Overflows

You are the security administrator for FrobozzCo, a large corporation with a great many secrets. You have just come back from a much-needed four week vacation in West Shanbar, only to find that FrobozzCo has been having some serious security issues! Prior to you joining FrobozzCo, you had taken some university computer security courses. In order to do everything you need, you have prepared a test environment on SPHERE with the software installed.

4.1.1 Initial setup

You can start the single-host SPHERE setup using the bufferoverflow lab. You will need to customize it a little.

Make sure that you save your work as you go. See the instructions in the submission section of this exercise for information about save and restore scripts. Make sure that you save any changes you make to the sourcecode, your patches, memos, etc. in your home directory so they are not lost when you swap out your experiment.

FrobozzCo has a longstanding tradition of reinventing the wheel whenever possible. As the old saying goes, "Why use something great that someone else made when you can use a mediocre thing you made yourself?" Additionally, the prevailing belief in management is that in-house software is more secure than third-party software since FrobozzCo alone has access to the source code. To that end, back when the Great Web Revolution hit and statistics relating to frobnick production were needed by remote facilities, the higher-ups at FrobozzCo insisted that their engineers write a webserver daemon, and it has been dutifully (if unspectacularly) serving web pages for many years.

Unfortunately, it is clear that someone has "rooted" (i.e., gained unauthorized superuser access to) the server; a number of root access files were copied out over the Internet and then the server started sending tons of spam. Fortunately, no data was lost, but the intruder had full control of the server and it is still unknown how they got in. You are convinced there is an exploitable buffer overflow bug in the web server software, but your boss, William H. Flathead III laughed off your suspicions saying, "I wrote the web server software – and I would never have made that mistake!" Nevertheless, he suggests you investigate the possibility of a buffer overflow, "just to be sure."

He asks you to produce a one page memo with an attached **working demo** that targets a specific buffer overflow (should one exist) causing the server to crash. This should be an intentionally exploitable hole in the code and not simply a robustness issue. If you find a vulnerability, your boss wants a patch to fix it. Finally, he wants to know how to clean up this mess – how severe is this specific compromise? How can we restore the system to a safe state?

4.1.2 Buffer overflows on the webserver

1. Load your experiment in SPHERE. Access the bufferoverflow node.
2. Find the webserver code, located in `/usr/src/fhttpd`.
3. Compile and run the code, using the instructions in the lab manual section on compiling and running the webserver.
4. Copy `webserver.c` to `webserver.orig.c` so that you can make a patch against the original.

5. Find the buffer overflow in the flhttpd webserver code.
6. Exploit the overflow, causing the software to crash.
 - Please note: you may be able to crash the software in other ways (e.g., a null pointer dereference) – we are only specifically interested in a buffer overflow caused by input that is not properly bound-checked.
 - See the section on the HTTP protocol, where you can learn about what the HTTP API looks like and how to send commands to web servers manually.
7. Create your exploit program, using the skeleton exploit we provided in `/root/submission/exploit.sh` and `/root/submission/payload`. Edit `/root/submission/payload` to include your attack data.
8. Fix the buffer overflow bug in the flhttpd sourcecode and create a patch against the original using the information below (and in the lab manual section on diff and patch).
9. Once you have fixed the flaw (and assuming you have `webserver.orig.c`), create a patch like so:
 - `cd` into the source code directory
 - execute:

```
diff -Naur webserver.orig.c webserver.c > webserver.patch
```
10. Write a 1 page memo:
 - (a) Describe the security flaw you found, how you fixed it, and how your demo exploit works. (The memo itself should quote as little sourcecode as possible; for longer sections, refer to filenames and line numbers in the original or your attached patch.)
 - (b) Considering flhttpd alone, include in your memo:
 - i. An evaluation of the seriousness of the breach
 - ii. A recovery plan for the server. (Is it enough to fix the flaw? Why or why not?)
 - iii. Any other observations or thoughts you might have.
11. Put the following files into in `/root/submission`:
 - (a) your memo
 - (b) your working demo with instructions
 - (c) your patch
12. Use the scripts described in the submission section for creating a submission tarball.

4.2 Bonus - Remote Execution

For the buffer overflow, the assigned task is to simply crash `fhttpd` with an attack payload. However, it is possible to inject code via the payload and not crash `fhttpd` but take over the application. Since `fhttpd` runs with root privileges, you can execute anything on the system if you manage to take over the program. For extra credit, do 1 and/or 2 below, and then 3:

1. Create and successfully execute a remote execution exploit using the vulnerability in `fhttpd`. This could be as simple as writing a file to the system.
2. Own the box: Create a remote shell you can send commands to, add a login user with root access, or some other reliable remote access facility.
3. Draft a short writeup describing the steps you took to create the exploit and what you learned in the process.

4.3 Tar ball creation

For this exercise, you will submit a tarball containing your patch, memo, and exploit code. Use the script `submit.sh` in `/root` on the server host for creating and restoring those tarballs.

submit.sh and restore.sh

`submit.sh` will back up:

- the source code directory `/usr/src/fhttpd/`
- everything in `/root/submission`, which should include:
 1. `exploit.sh`
 2. `payload`
 3. your memo
 4. patch

`restore.sh` will restore those files to their original locations, automatically overwriting whatever is there.

5 Deliverables

1. A report describing all your findings above.
2. A zip file containing:
 - The tar ball that resulted from your data collection listed above.
3. Submit by the class on April 30, 2025.

6 Grading

Points will be subtracted if any of the pieces of the deliverables are missing or incomplete. The late submission policy applies.